

Intro to CNN

CSCI 611 Sp25 Week 3 Session 3

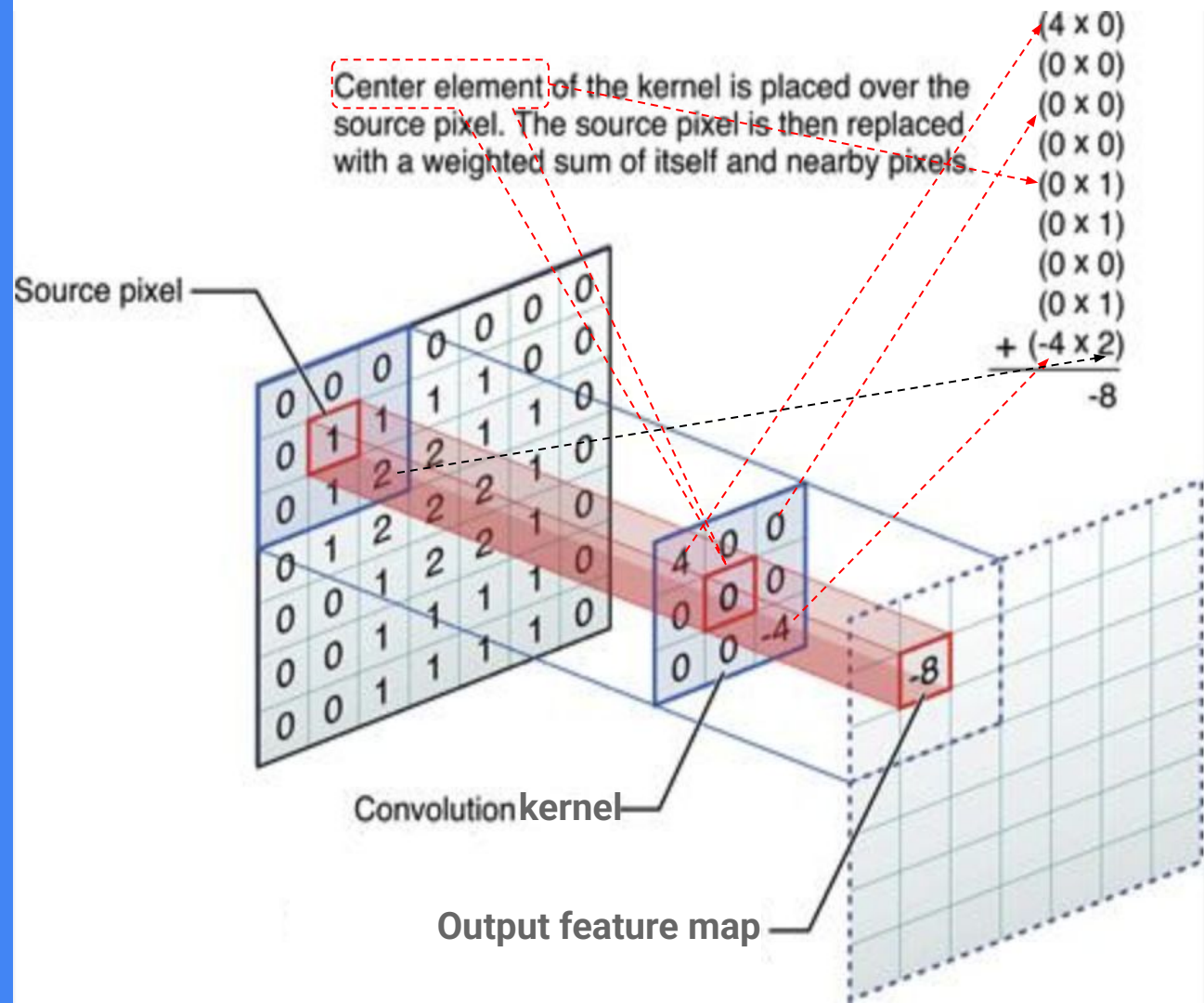


Image Filtering through Convolution

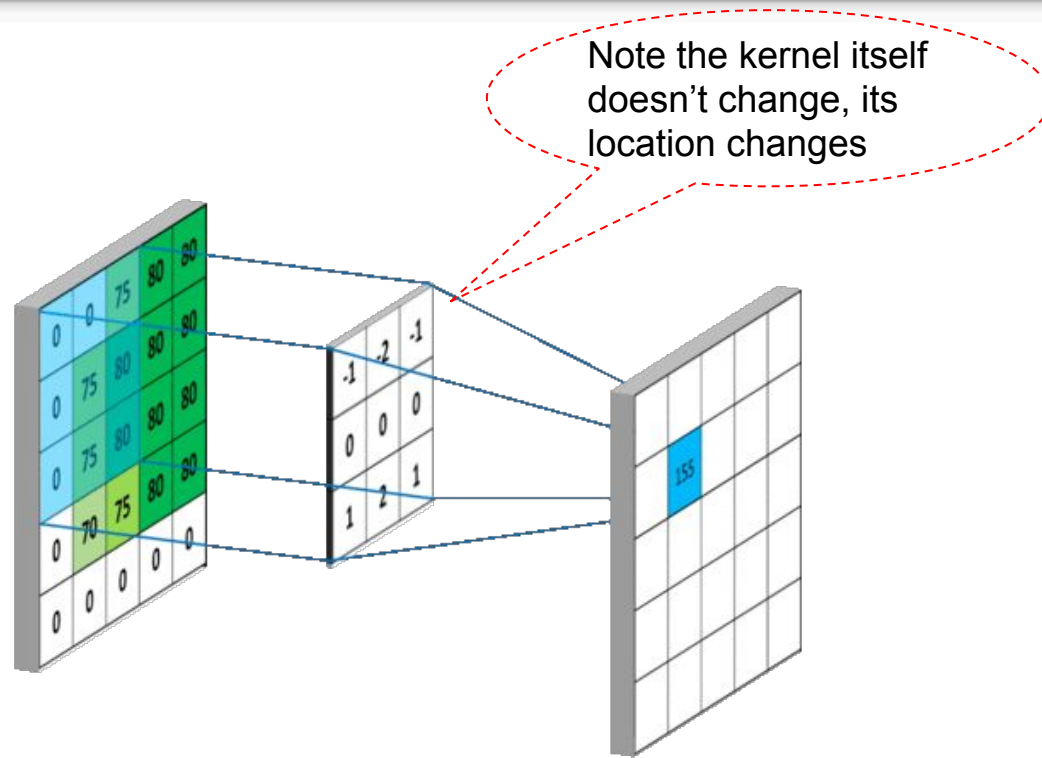
Take a **local** region of interest (3x3 in this example) from the source image, project a **weighted sum** to the output

Slide the kernel through the source image and compute the **dot product** from the kernel elements (**weights**) and the corresponding source pixels.

Kernel = Mask = Filter



Convolution In Action

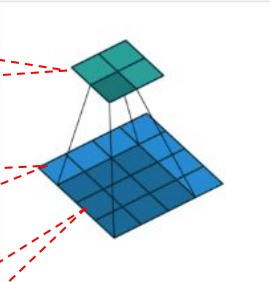
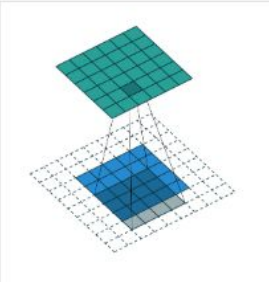
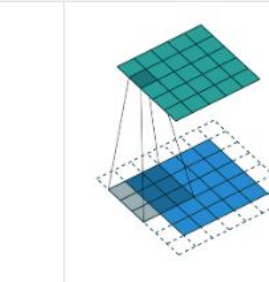
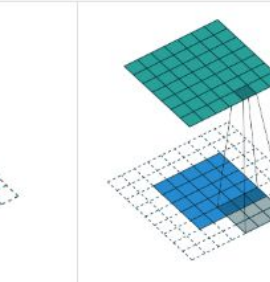
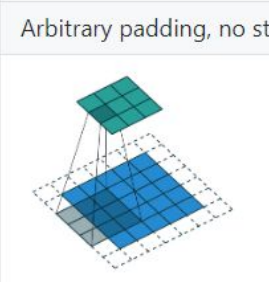
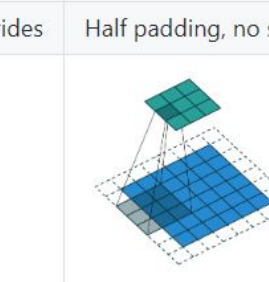


Visualize Convolution

https://github.com/vdumoulin/conv_arithmetic/blob/master/README.md

Convolution animations

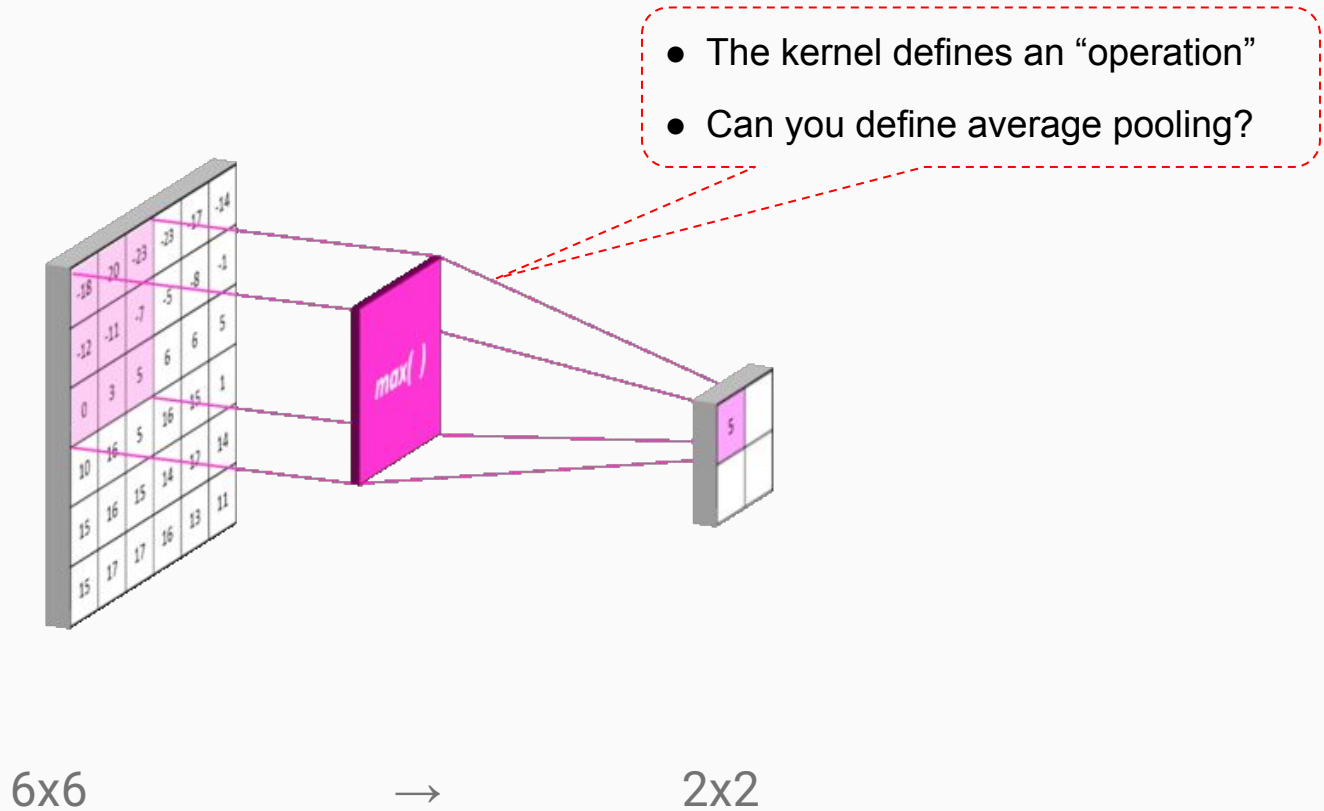
N.B.: Blue maps are inputs, and cyan maps are outputs.

 output (2x2) input (4x4) convolution kernel/mask (3x3)			
No padding, no strides	Arbitrary padding, no strides	Half padding, no strides	Full padding, no strides
			
No padding, strides	Padding, strides	Padding, strides (odd)	

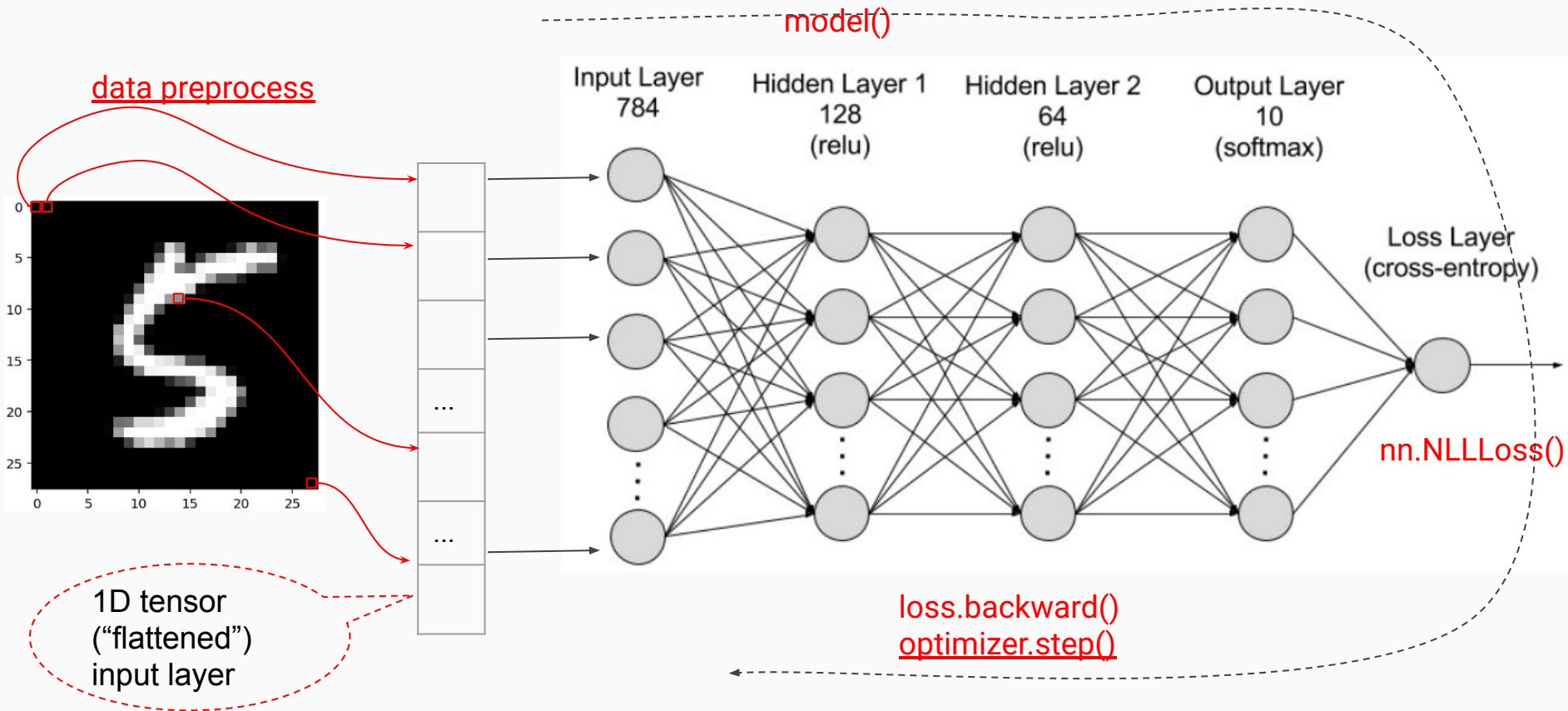
No strides means
stride=1 → slide
kernel 1 pixel at a
time

Strides means stride
> 1 → slide kernel by
skipping pixels from
input

Max Pooling in Action



Build Model Using Pytorch



Multilayer Perceptron (Dense hidden layers)

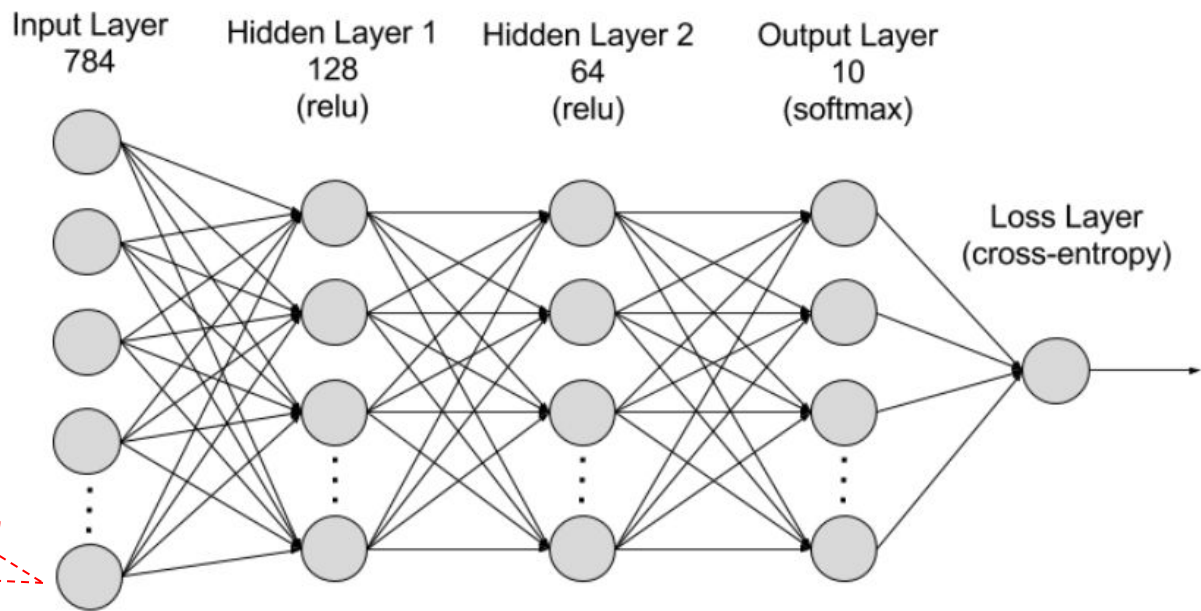
Number of weights

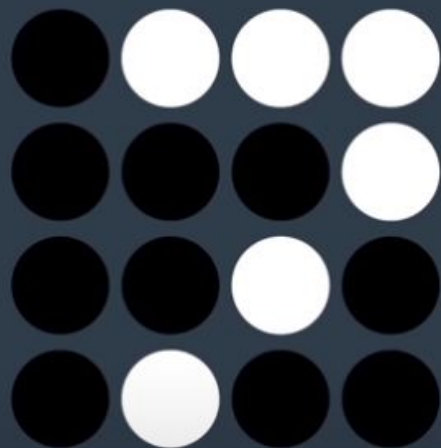
$$=(784+1) \times 128 + (128+1) \times 64$$

$$+(64+1) \times 10$$

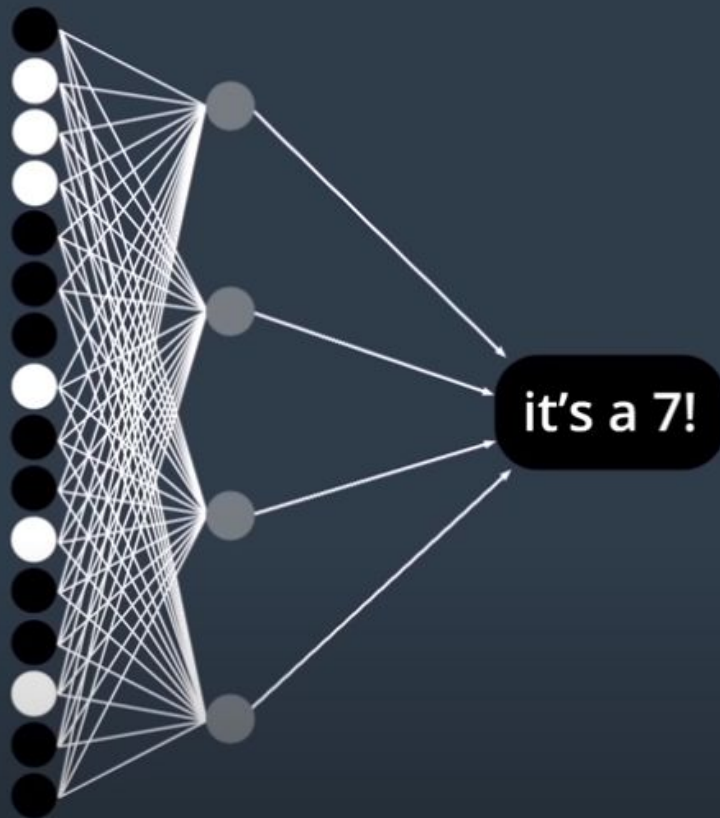
$$= 109386$$

1D tensor
("flattened")
input layer



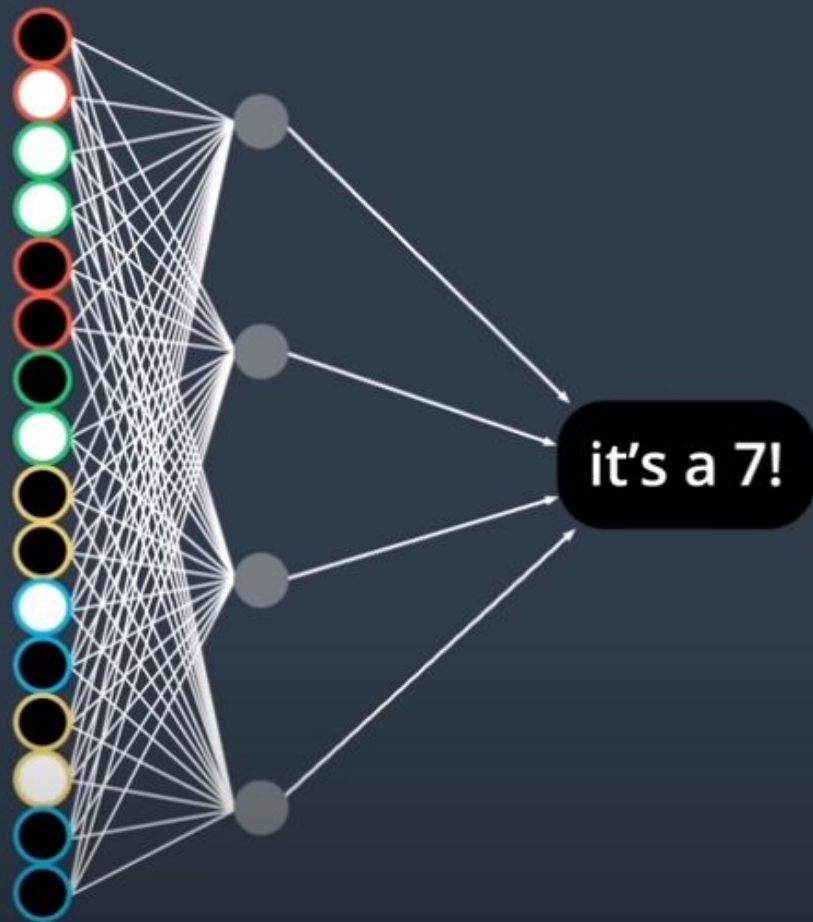


flatten
→



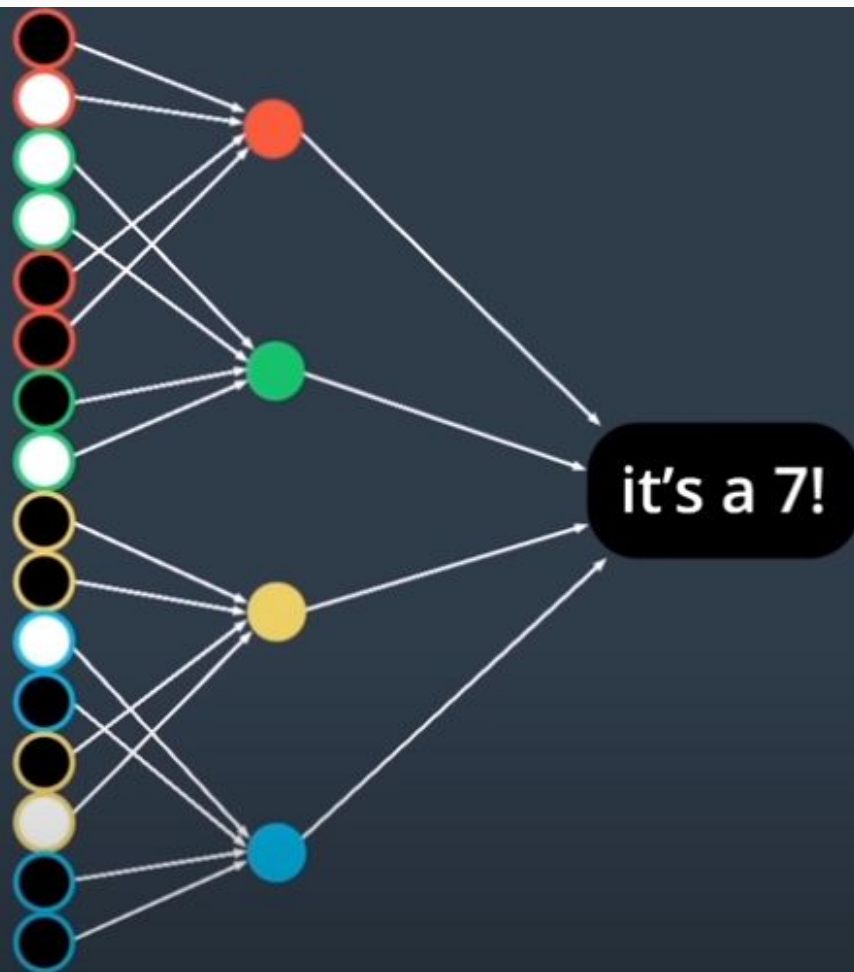


flatten
→

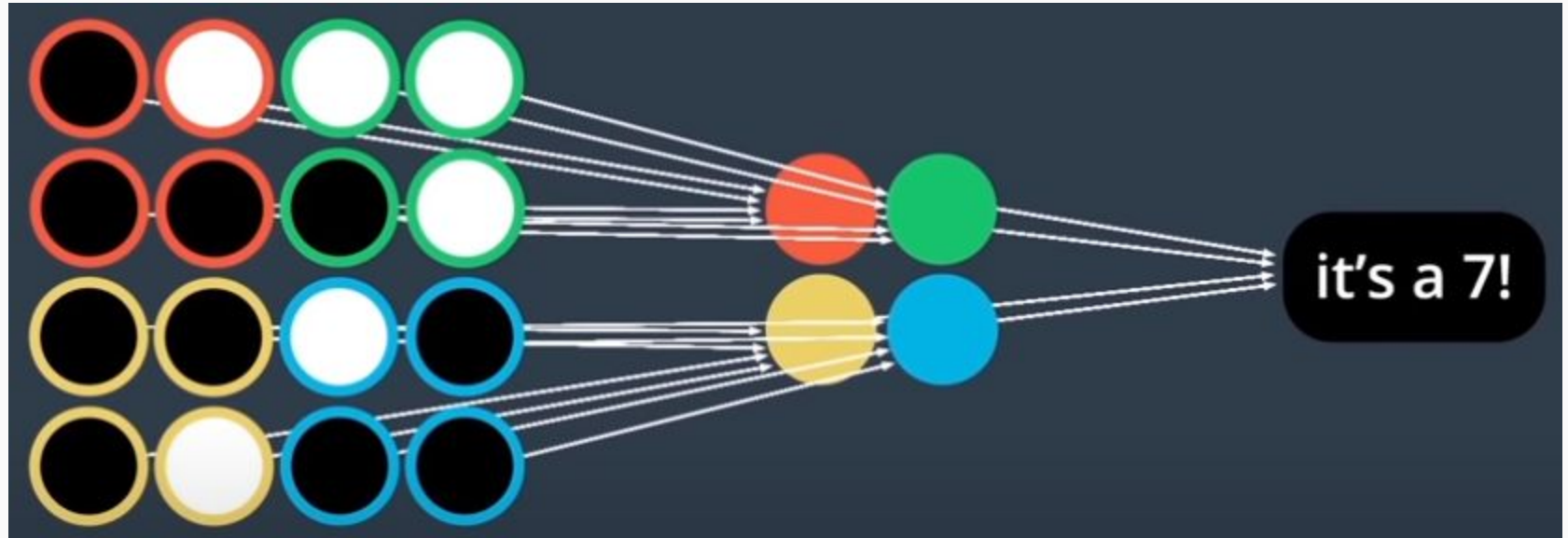




flatten



Spatial Correlation







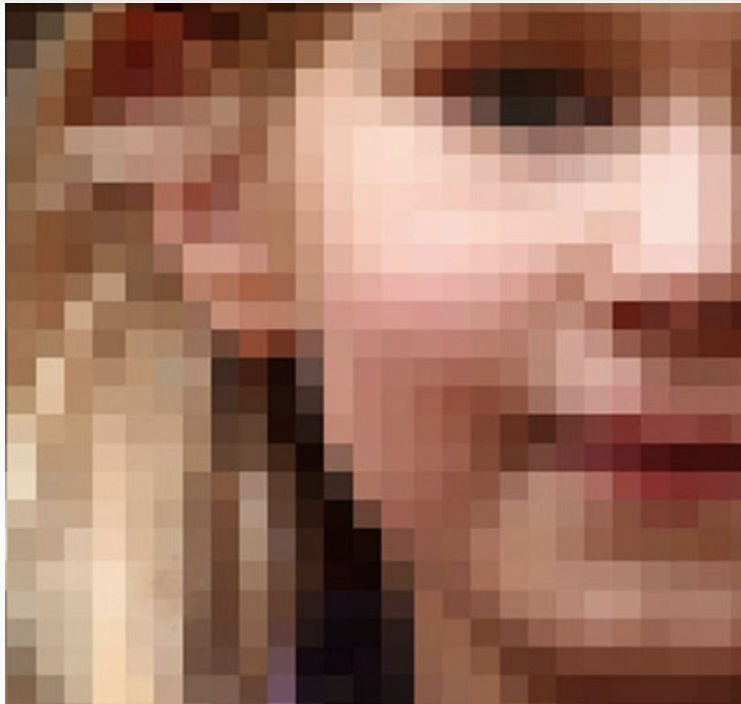
Maxpooling – Global Context



Max Pooling – Global Context



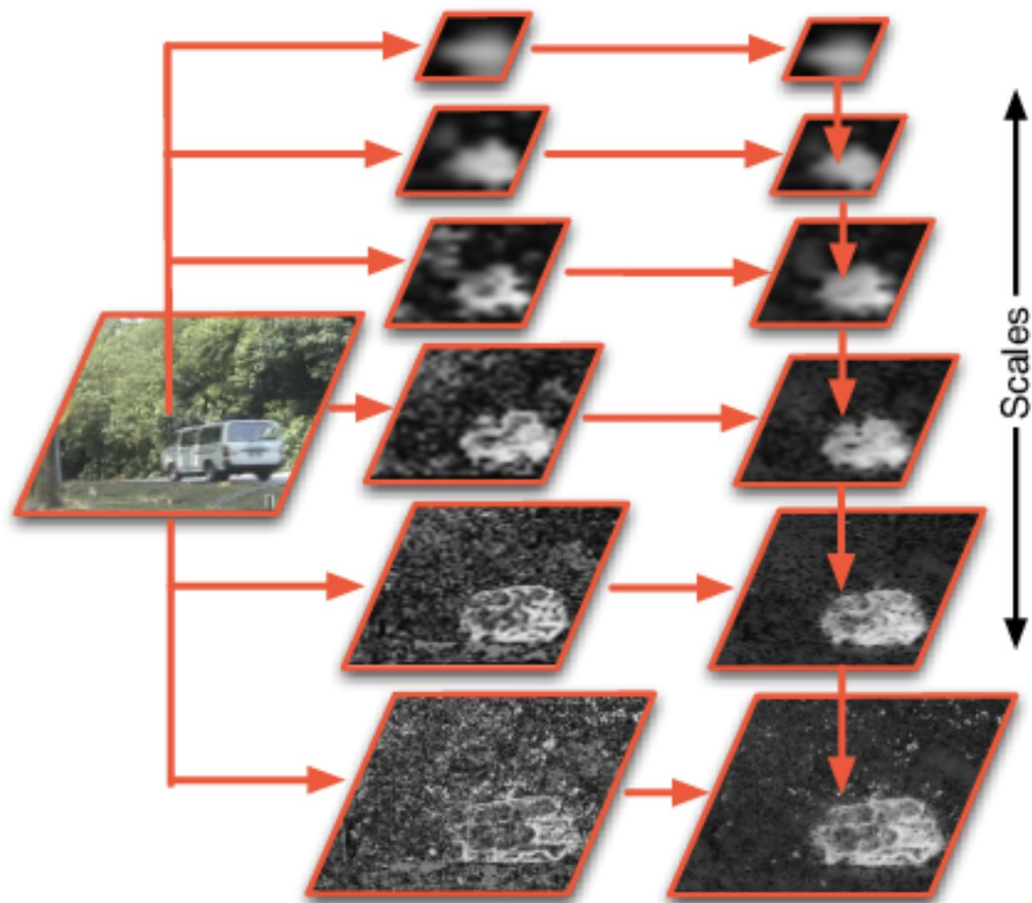
Max Pooling $\rightarrow$ Lower Spatial Resolution
Is “classification” still possible?



Why Max Pooling?

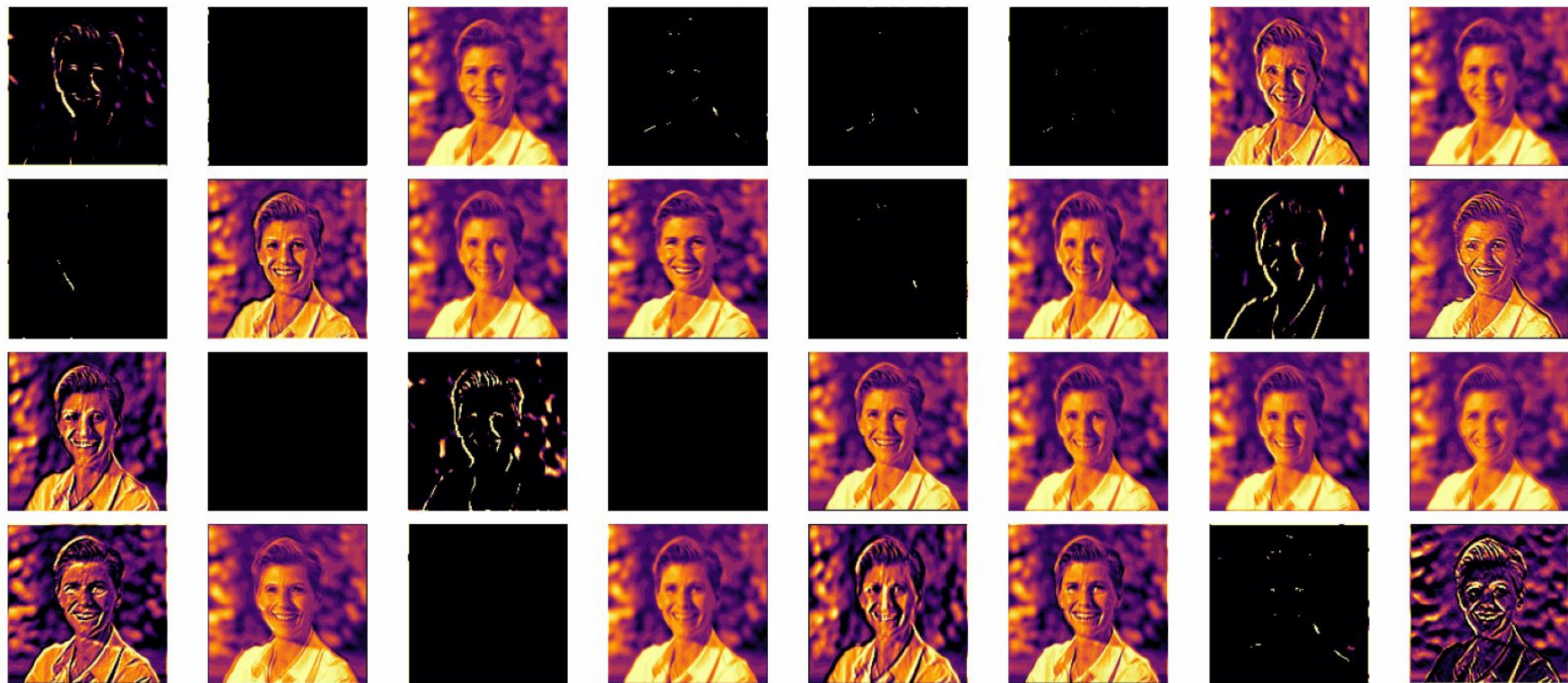
- **Increased receptive field:** higher layers “see” larger regions of the input
- **Salient feature selection:** keeps the strongest activation (e.g., edge, corner) and suppresses noise
- **Regularization effect:** reduces overfitting by discarding precise spatial details
- **Dimensionality reduction:** reduces spatial size → fewer parameters and faster computation

Multi-Scale Image Analysis

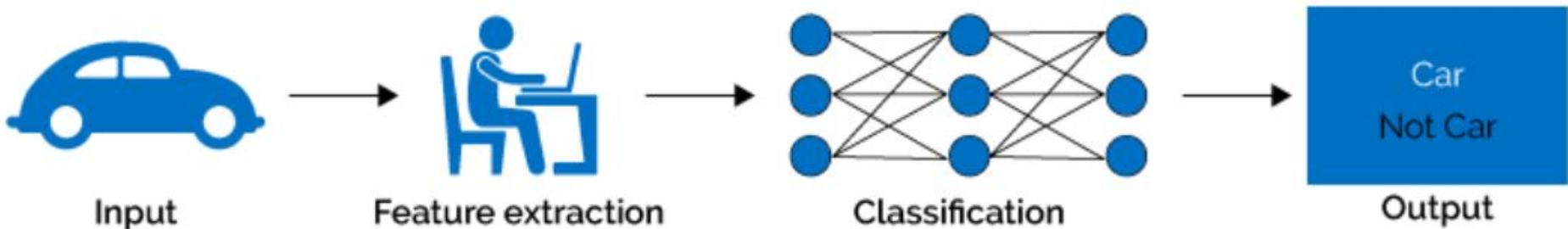


Convolution Layer Feature Extraction (from I. Miziuk)

Layer: Block1_Conv1 | Epoch 1



Machine Learning



Deep Learning (CNN)

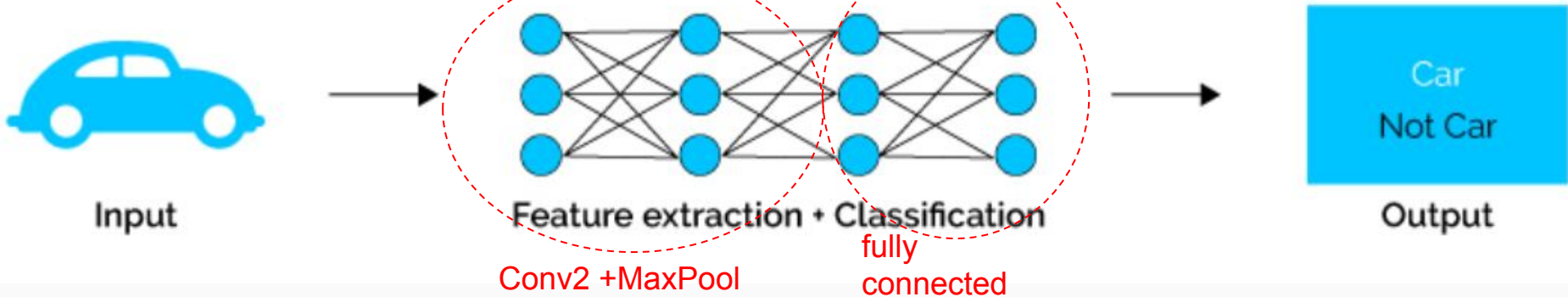
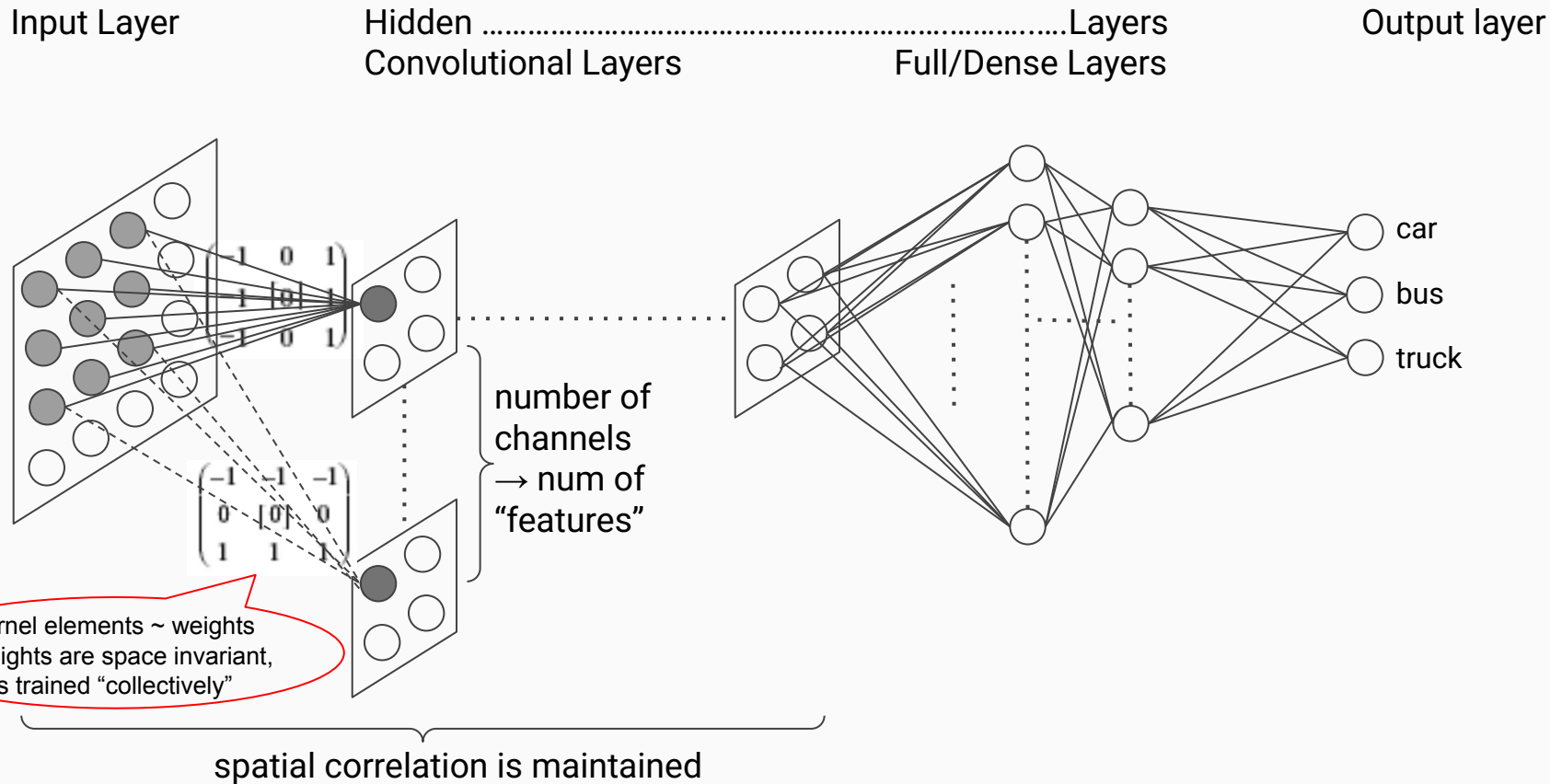


Image Filtering Kernel to Weights in CNN



Example Code – CNN

```
class OurConvNet(torch.nn.Module):
    def __init__(self, num_outputs=20):
        super(OurConvNet, self).__init__()
        self.Conv1 = nn.Conv2d(3, 64, 5, 1, 2)
        self.Sigma = nn.Sigmoid()
        self.Avg = nn.AvgPool2d(2, stride=2, padding=0)
        self.Conv2 = nn.Conv2d(64, 128, 5, 1)
        self.Fl = nn.Flatten()
        self.Linear = nn.Linear(128*5*5, num_outputs)

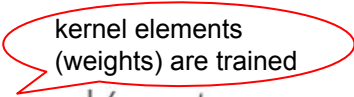
    def forward(self, x):
        out = self.Sigma(x)
        out = self.Conv1(out)
        out = self.Avg(out)
        out = self.Conv2(out)
        out = self.Avg(out)
        out = self.Fl(out)
        out = self.Linear(out)
        return out
```


Convolution Layer

<https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

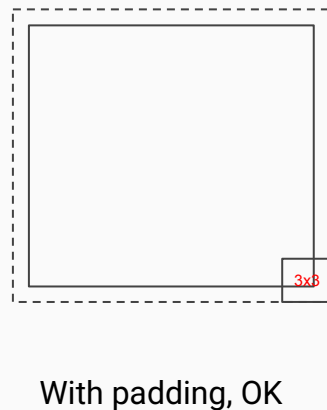
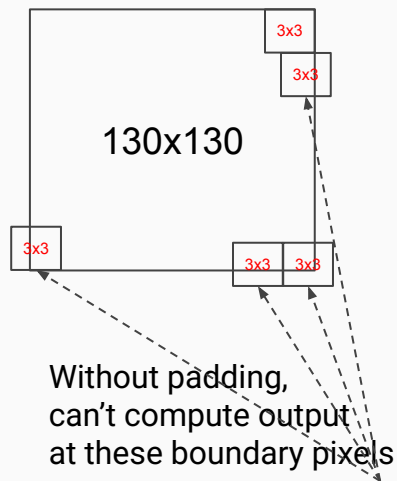
We typically define a convolutional layer in PyTorch using `nn.Conv2d`, with the following parameters, specified:

```
nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)
```

- `in_channels` refers to the depth of an input. For a grayscale image, this depth = 1
- `out_channels` refers to the desired depth of the output, or the number of filtered images you want to get as output
- `kernel_size` is the size of your convolutional kernel (most commonly 3 for a 3x3 kernel)

- `stride` and `padding` have default values, but should be set depending on how large you want your output to be in the spatial dimensions x, y

Kernel size, Stride, Padding

- Stride can't be 0, kernel needs to move (slide)
- No stride (stride=1) with half padding means no downsampling
- No padding (padding=0) means discarding some boundary pixels
- Question:
 - If kernel size=7, what is the padding required to produce same size (wxh) output?



Pooling Layer

Maxpooling layers commonly come after convolutional layers to shrink the x-y dimensions of an input

- No weights to train
- Number of channels (features) doesn't change
- $\text{kernel_size}=\text{stride}=k \Rightarrow$ downsample by a factor of k

<https://pytorch.org/docs/stable/generated/torch.nn.MaxPool2d.html>

```
CLASS torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1,
                           return_indices=False, ceil_mode=False)
```

[SOURCE]

Applies a 2D max pooling over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C, H, W) , output (N, C, H_{out}, W_{out}) and `kernel_size` (kH, kW) can be precisely described as:

$$\text{out}(N_i, C_j, h, w) = \max_{m=0, \dots, kH-1} \max_{n=0, \dots, kW-1} \text{input}(N_i, C_j, \text{stride}[0] \times h + m, \text{stride}[1] \times w + n)$$

If `padding` is non-zero, then the input is implicitly zero-padded on both sides for `padding` number of points. `dilation` controls the spacing between the kernel points. It is harder to describe, but this [link](#) has a nice visualization of what `dilation` does.

The parameters `kernel_size`, `stride`, `padding`, `dilation` can either be:

- a single `int` – in which case the same value is used for the height and width dimension
- a `tuple` of two ints – in which case, the first `int` is used for the height dimension, and the second `int` for the width dimension

Example Conv/Max Pooling layers

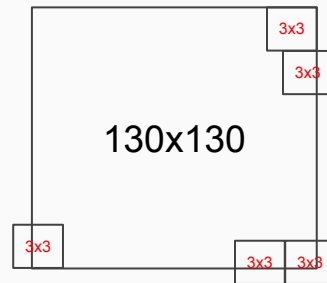
Consider an input image that is 130×130 (x, y) and 3 in depth (RGB). Say, this image goes through the following NN layers in order:

`nn.Conv2d(3, 10, 3)` $\Rightarrow$ padding=0

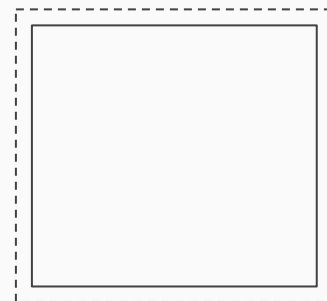
`nn.MaxPool2d(4, 4)`

`nn.Conv2d(10, 20, 5, padding=2)`

`nn.MaxPool2d(2, 2)`

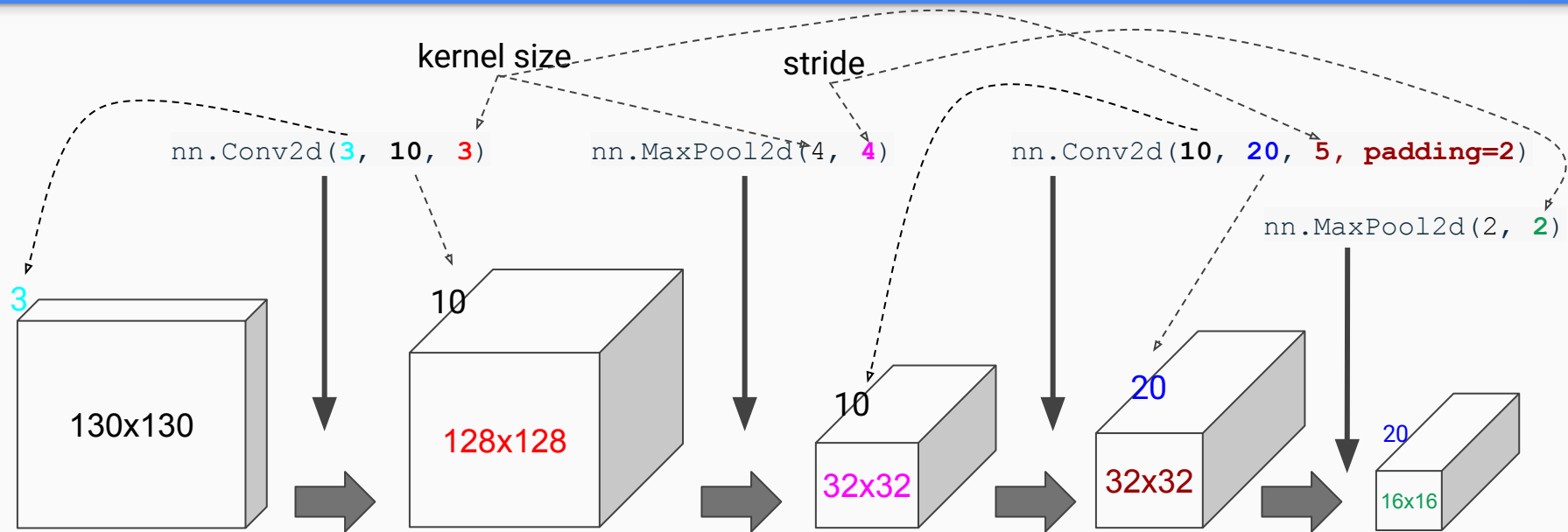


no padding



with padding

Visualize Conv/Max-Pooling Layers

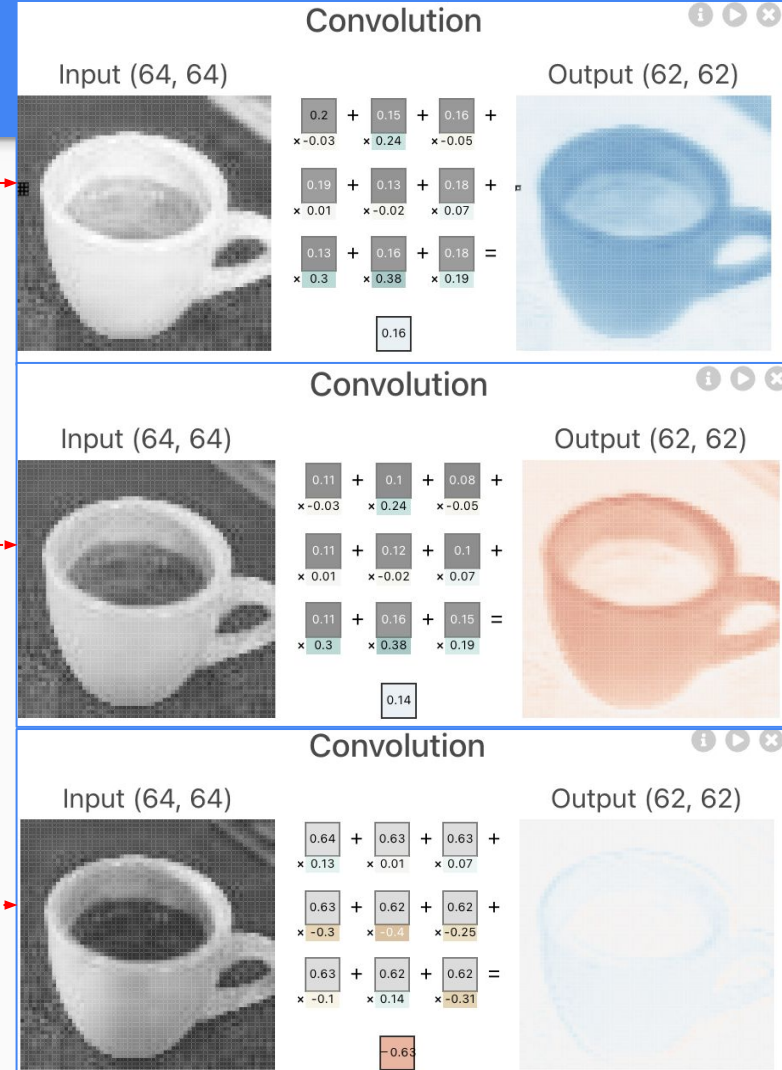
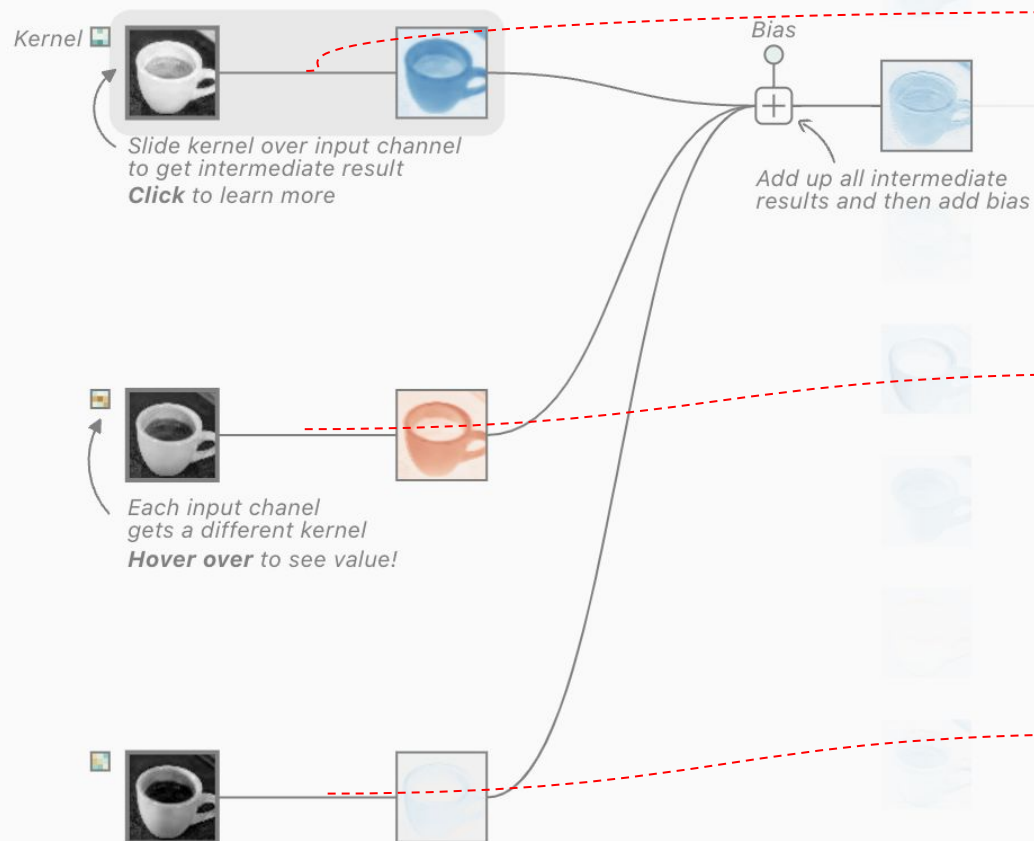


Visualize CNN

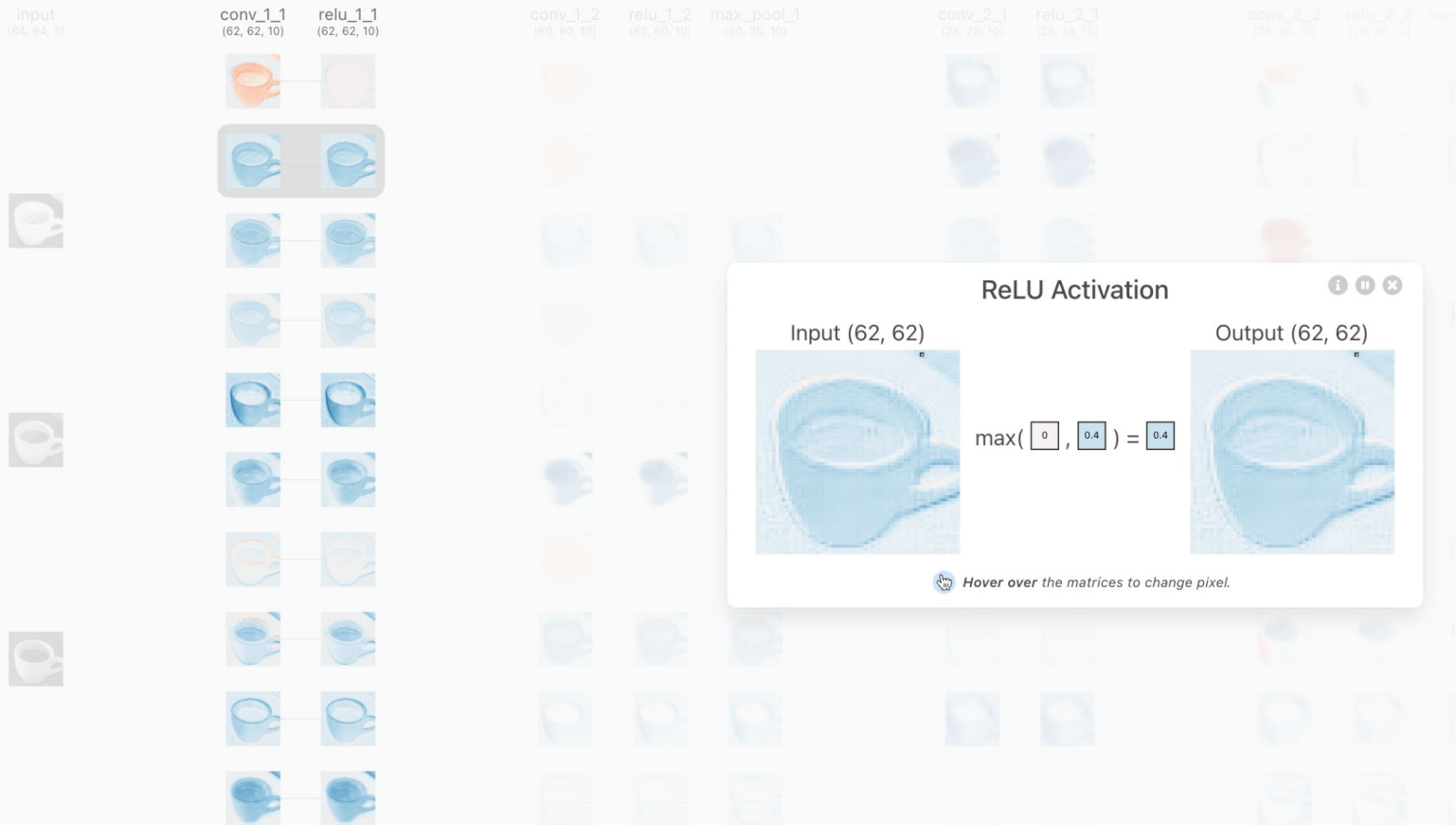
More screenshots from:

<https://poloclub.github.io/cnn-explainer/>

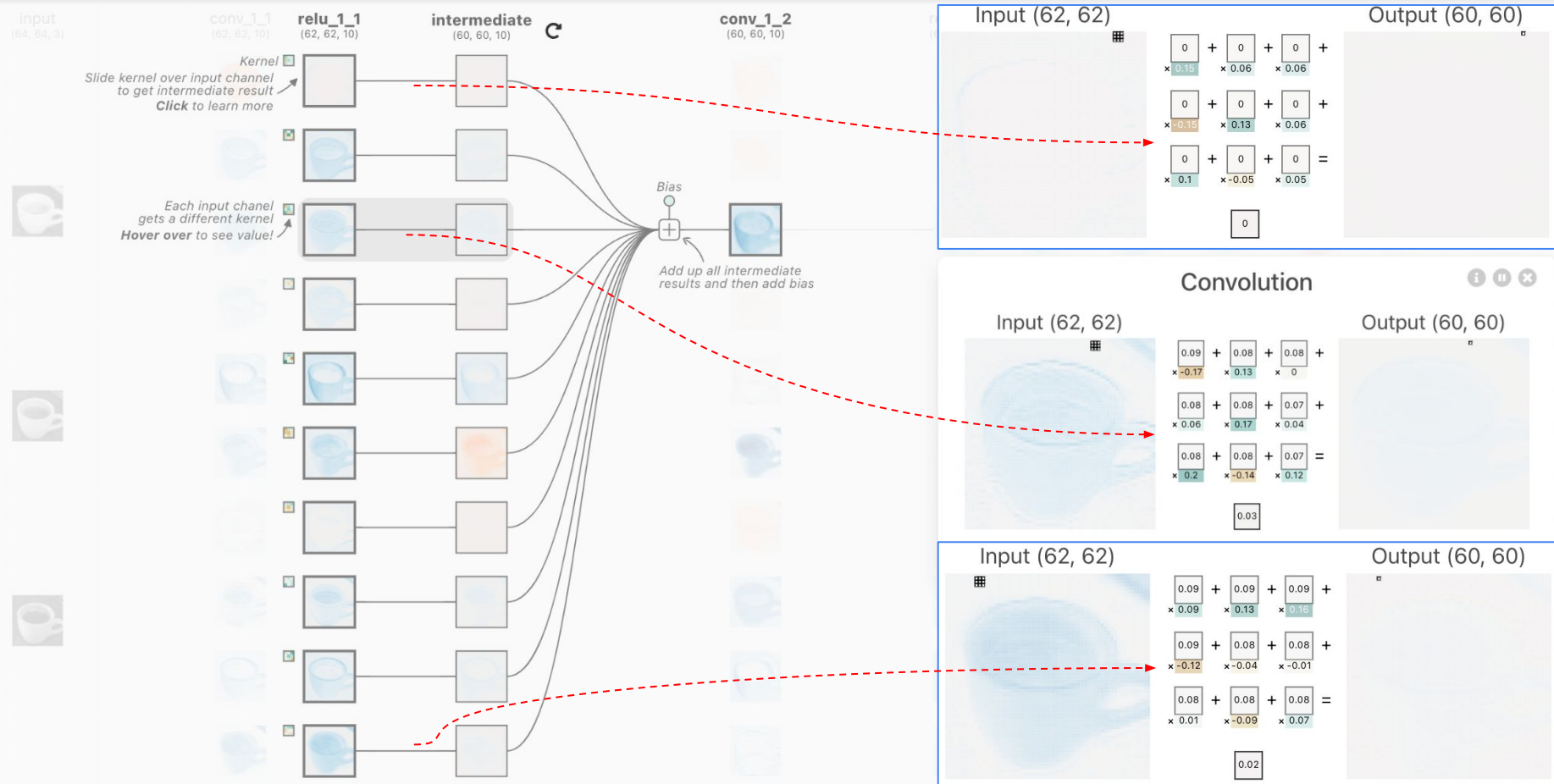
CNN Visualizer (1st conv layer, 3 in→10 out)



Activation (Layer)



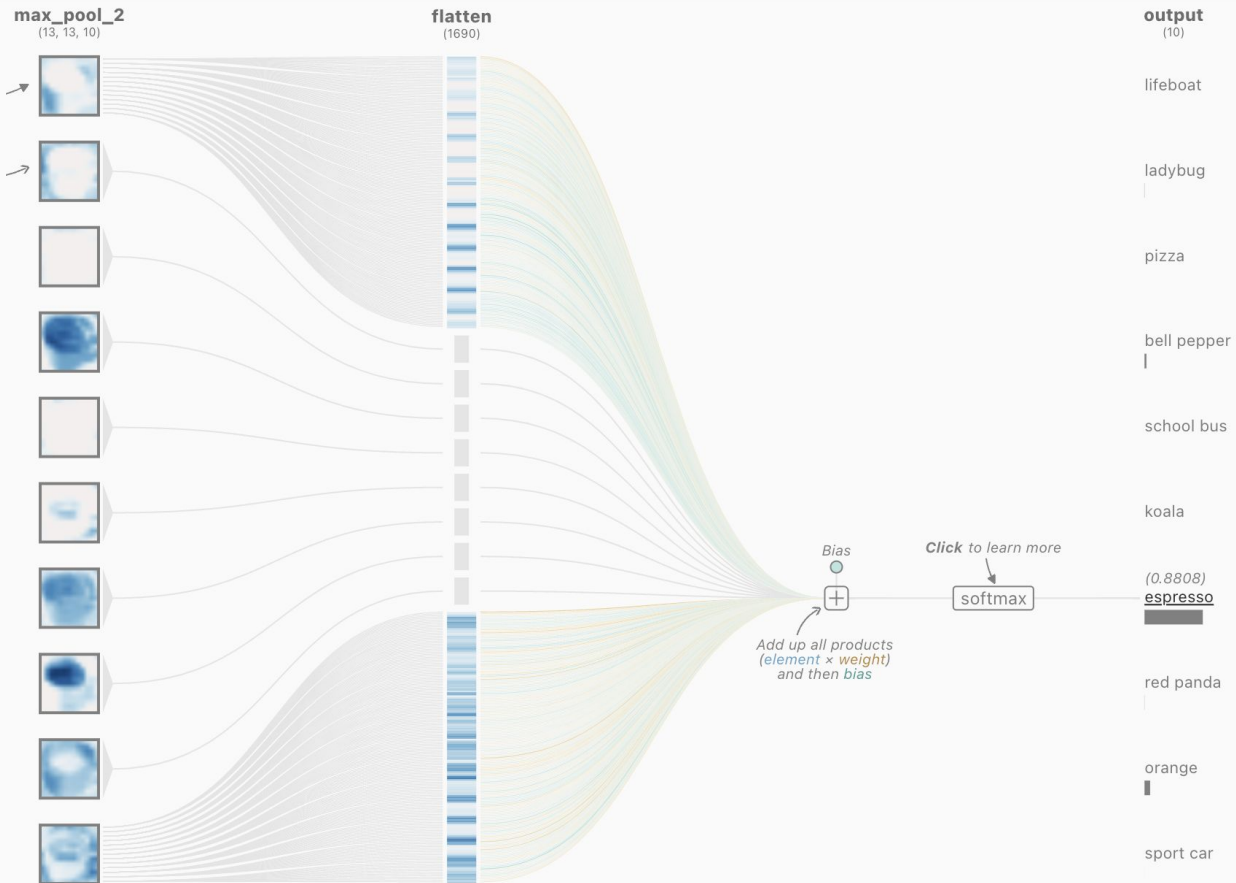
2nd Convolution Layer, 10 in $\rightarrow$ 10 out [10 kernels for the 3rd output channel in conv_1_2]



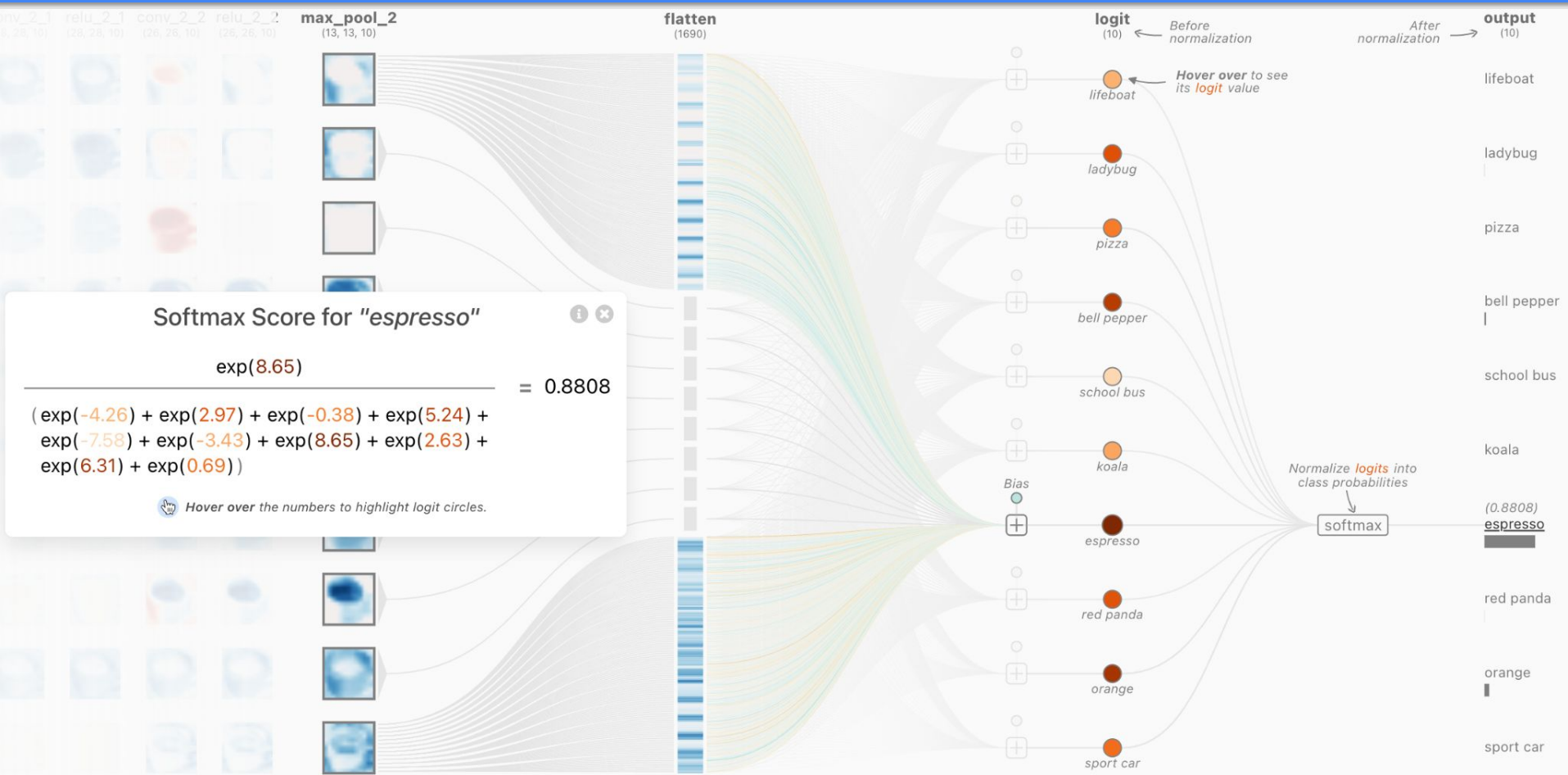
MaxPooling Layer



Fully Connected Layer



Output (Softmax) Layer



Q&A